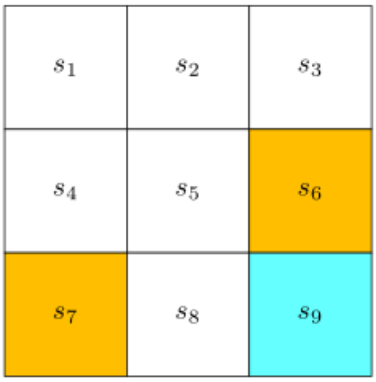
1 Mathematical Foundations of RL – Shiyu Zhao

2025, Springer nature, Singapore. 283 pages. Read until page: 22
https://github.com/MathFoundationRL/Book-Mathematical-Foundation-of-Reinforcement-Learning

1.1 Basic Concepts



The robot is the agent. The agent goal is finding a good policy to reach the target cell.
State: the agent status wrt the environment. Example. 9 cells => 9 possible states.
State space= $S=\{s_1, s_2, \ldots, s_9\}$

s_6 and s_7 are forbidden cells

Action space = $A=\{a_1, a_2, a_3, a_4, a_5\}$. The set of all possible actions. Moving upward, rightward, downward, leftward, staying still.
Different states can have different action spaces. In the book, all actions are considered.
State transition: when taking an action, the agent can move from a state into another.

$s_1 \xrightarrow{a_2} s_2$

When an agent tries to go beyond a boundary, it is bounced back, so

$s_1 \xrightarrow{a_1} s_1$: the agent tried to move upward.

Forbidden cells? For example s_6 . Two scenarios-

Scenario 1: forbidden cells are accessible: $s_5 \xrightarrow{a_2} s_6$

Scenario 2: forbidden cells are not accessible: $s_5 \xrightarrow{a_2} s_5$: the agent is bounced back by the cell s_6 .

The book uses the scenario 1 : forbidden cells are accessible, but the agent is punished when stepping into them.
The state transition process can be represented as a table.

	a_1 **Up**	a_2 **Right**	a_3 **Down**	a_4 **Left**	a_5 **Still**
s_1	s_1	s_2	s_4	s_1	s_1

Expressed as probabilities: $p(s_1|s_1,a_2)=0$ $p(s_2|s_1,a_2)=1$

The table representation can only describe deterministic state transitions.
In general, state transitions can be stochastic and must be described by conditional probability distributions.
Policy: which action to take at every state. Following a policy, the agent can generate a trajectory from an initial state.
Policies can be described by conditional probabilities.
A policy is a probability distribution, denoted as $\pi(a|s)$, defined for every state: it indicates the probability of selecting each action. For example: $\pi(a_2|s_1)=0.5$ $\pi(a_3|s_1)=0.5$, all other actions have zero probability. Policies represented by conditional probabilities can be stored as tables. The item at row i and column j is the probability of taking the action a_j while in the state s_i

Reward: $r(s|a)$ depends on taking an action while in a state. The reward can be positive, negative, or zero. A positive reward encourages the agent to take that action.. Rewards in the agent world.

- If an agent tries to exit the boundary: $r=-1$
- If an agent tries to enter a forbidden cell $r=-1$
- If an agent reaches the target state: $r=+1$
- Otherwise, no reward. $r=0$

2 Deep Reinforcement Learning Course – Hugging Face

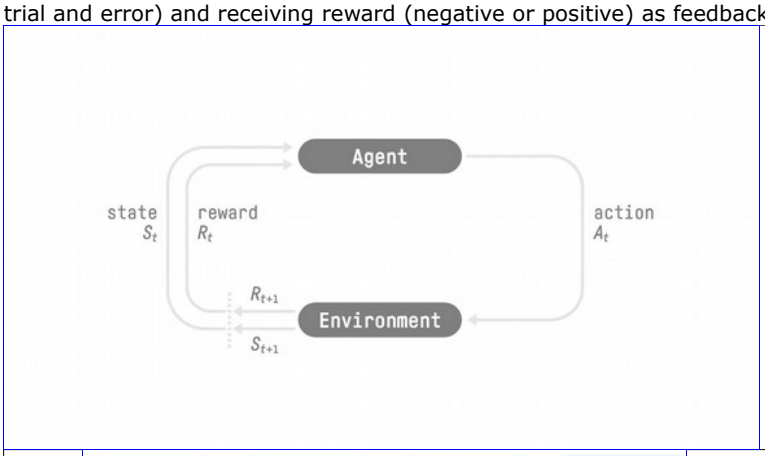
Link: https://huggingface.co/learn/deep-rl-course/
Hugging Face tarpit.xyz@gmail.com Passwd: SpedHfabHg23* Username: Mingux
Token. Token name: DRL_Course Role: write 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
Google Colab: https://colab.research.google.com/ diego.minguz@gmail.com
Discord: https://discord.com/ tarpit.xyz@gmail.com Username: diegominguzzi Password: *SpesDabHg
Discord channels to watch

- rl-announcements: where we give the lastest information about the course (link)
- rl-discussions: where you can exchange about RL and share information.
- rl-study-group: where you can ask questions and exchange with your classmates.
- rl-i-made-this: where you can share your projects and models.

2.1 Unit 1 – Introduction to Deep Reinforcement Learning



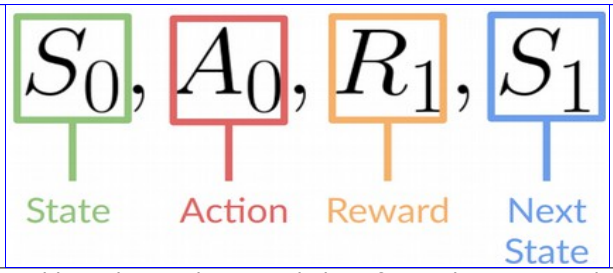
https://huggingface.co/learn/deep-rl-course/unit1/
Stable-Baselines3 library : https://stable-baselines3.readthedocs.io/en/master/
The idea behind Reinforcement Learning is that an agent (an AI) will learn from its environment by interacting with it (through trial and error) and receiving reward (negative or positive) as feedback for performing actions.



Formal definition of RL: Reinforcement learning is a framework for solving control tasks (also called decision problems) by building agents that learn from the environment, interacting with it through trial and error and receiving rewards (positive or negative) as unique feedback.
RL process: a loop of: state, action, reward and next state.
At the time `t`,

- the agent receives current state and reward
- the agent executes an action

At the time `t+1`, the agent receives the new state and new reward, depending on the previous state and previous action.



The loop is called **SARS**.
The RL loop outputs a sequence of state, action, reward and next state.

The reward hypothesis: the central idea of RL. The agent goal is the maximization of the expected cumulative reward.
The Markov property: the agent needs to know only the current state to decide the next action, not the history of all actions and states.
Distinction between observation and state space. State: a complete description of the world, there is no hidden information. For example: in chess, it is enough to look at the board to know the state of the game. Observation: a partial description of the world.


Chess: the state of the world



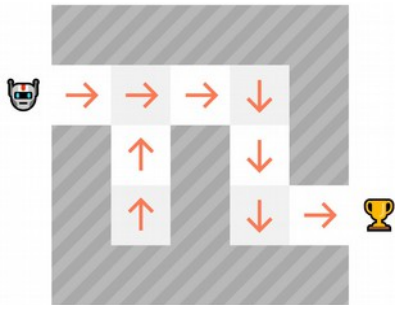
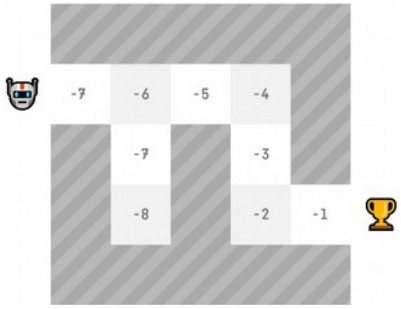
Mario: an observation of the world

The action space is the set of all possible actions in an environment. Actions can be discrete (a finite set) or continuous (an infinite set).

Rewards and discounting Reward without discount factor. $R(\tau)=\sum_{k=0}^{\infty} r_{\tau+k+1}$. The discount factor decreases how much the agent cares about the long term reward. $R(\tau)=\sum_{k=0}^{\infty} \gamma^k r_{\tau+k+1}$ The lower the γ^k, the more short-term minded the agent is.

Types of tasks: episodic: there is a starting and ending point; for example a Mario Bros episode. Continuing tasks: these tasks continue forever. Example stock trading: the agent must learn while interacting with the environment.
Exploration/exploitation tradeoff. Exploration: trying new actions to find more information about the environment. Exploitation: using known information to maximize reward. We must define a rule that helps to handle this trade-off.
The Policy is denoted as π $Action = \pi(State)$. The policy outputs an action or a probability $p(a)$ distribution over the actions. We must find the optimal policy π^*. Two approaches to find π^*:

- **Policy-based methods**. Directly: teaching the action to take given the current state.
- **Value based methods**: teach the agent to learn which state is more valuable, then take the action that leads to the more valuable states.

Policy based methods	Value-based methods
	
Define a mapping from state to action, or from state to a probability distribution over all possible actions in that state.	The value function defines a value for each state.

Policy based methods:

- Deterministic policy: $a = \pi(s)$. The agent always picks the same action in a given state.
- Stochastic policy: $\pi(a \mid s) = P[A \mid s]$ probability distribution over the actions, given the state. Example: Left:`0.1`, Right:`0.7`, Jump:`0.2`.

Value based methods: we learn a value function, that maps a state to the expected value of being in that state. The value of a state is the expected discounted return, that the agent gets if it starts in that state and acts according to our policy π.
"*Acts according to our policy*" means: going to the state with the highest value.

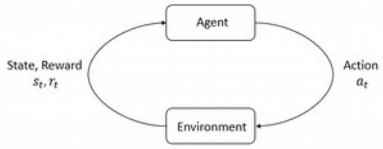
$$\text{Value function calculated in } s: \; v_\pi(s) = E_\pi\left[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} \ldots \mid S_t = s \right]$$

Once defined the value for each state, at runtime the agent selects the action to reach the next state with the highest possibile value, as defined by the value function. The policy selects the action to reach the state with the biggest value, defined by the value function.
The "Deep" in DRL Q-learning is a classic RL algorithm, while Deep Q-learning uses a neural network to approximate the Q value.
Q-learning: the algorithm creates a Q-table. In classical RL, the algorithm creates the Q-table, in DRL, a neural network approximates the Q-table. See the summary of the unit and the glossary.
Exercise Unit 1:

- Environment: Lunar Lender:https://gymnasium.farama.org/environments/box2d/lunar_lander/

2.1.1 Notes from from the OpenAI site: Key Concepts in RL

- https://spinningup.openai.com/en/latest/user/introduction.html
- https://spinningup.openai.com/en/latest/spinningup/rl_intro.html



The cumulative reward is called **Return**.
A state is a complete representation of the world, an observation is a partial description of a state s. In DRL, states and observations are represented as tensors. When the agent observe the complete state of the environment, we say that the environment is fully observed, otherwise it is partially observed. The action space can be discrete or continuous.
Policies. They can be

- deterministic $a_t = \mu(s_t)$. Policy parameters (denoted as θ or ϕ) are written as subscript μ_θ
- stochastic $a_t \sim \pi(.|s_t)$

Stochastic policies:

- categorical policies: for discrete action spaces. It is a neural network that classifies over discrete actions. The input is the observation, the output are the probabilities of choosing each action; the layer before the output is probably a softmax.
- diagonal Gaussian policies are used in continuous action spaces

Important steps in selecting an action:

- sampling the actions from the policy
- computing the log likelihood of actions $\log(\pi_\theta(a|s))$

Trajectories: a trajectory $\tau = (s_0, a_0, s_1, a_1, \ldots)$ is a sequence of states and actions in the world. The state $s_{t+1} = f(s_t, a_t)$ only depends on the most recent state and action. Trajectories are also called episodes or rollouts.

Reward and return: $r_t = R(s_t, a_t, s_{t+1})$, often it is simplified as: $r_t = R(s_t)$, or $r_t = R(s_t, a_t)$.
The goal of the agent is that of maximizing the cumulative reward over a trajectory $R(\tau)$.
There is finite horizon undiscounted return $R(\tau) = \sum_{t=0}^{T} r_t$ and infinite horizon, with discounted return $R(\tau) = \sum_{t=0}^{\infty} \gamma^{t+1} r_t$.
The **RL problem**: *the goal of RL is to select a policy that maximizes the expected return when the agent acts according to it*.
Value functions: the expected reward when following the policy π.

Policy	**Start action: s**	**Chosen, arbitrary action** a
On-policy	On-Policy Value function $V^{\pi}(s)$	On-Policy Action-Value function $Q^{\pi}(s,a)$
Optimal	Optimal Value function $V^{*}(s)$, when acting according to the the optimal policy	Optimal Action-value function $Q^{*}(s,a)$

- $V^{\pi}(s) = E_{a\sim\pi}[Q^{\pi}(s,a)]$
- $V^{*}(s) = max_a Q^{*}(s,a)$

In on-policy formulas, the next action is sampled from the probabilities of each action in that state. In optimal, the action maximizes the reward. In state s, the optimal action is: $a^{*} = argmax_a Q^{*}(a,s)$.
$V(s)$ and $Q(s,a)$ calculate the return, not the reward.

2.1.1.1 Bellman Equations

They are recursive formulas. In case of the on-policy value functions.

- $V^{\pi}(s) = E_{a\sim\pi}[r(s,a) + \gamma V^{\pi}(s')]$, where s' is the next state
- $Q^{\pi}(s,a) = E_{s\sim P}[r(s,a) + \gamma E_{a'\sim\pi}[Q^{\pi}(s',a')]]$

In case of the optimal value functions:

- $V^{*}(s) = max_a E[r(s,a) + \gamma V^{*}(s')]$
- $Q^{*}(s,a) = E_{s\sim P}[r(s,a) + \gamma max_{a'}(Q^{*}(s',a'))]$

Bellman: value of a state= reward in a state + gamma * (Value of the next state).
The right-hand side of the formula is called Bellman backup.

2.1.1.2 Advantage Functions A(s,a)

Advantage functions: sometimes we do not describe how good an action is in absolute sense, but only how much better it is than others on average. The advantage function $A^{\pi}(s,a) = Q^{\pi}(s,a) - V^{\pi}(s)$. Note that $V^{\pi}(s)$ is the expected value, so it is the sum of all rewards starting from s, each weighted by its probability.

2.1.1.3 Formalism of the Markov decision process

- S : the set of all states
- A : the set of all actions
- $r_t = R(s_t, a_t, s_{t+1})$: the reward function
- $P(s' = s_{t+1} \mid a = a_t, s = s_t)$
- ρ_0 : the starting state distribution.

2.1.2 Kinds of Algorithms

https://spinningup.openai.com/en/latest/spinningup/rl_intro2.html
Model free and model based: model based if the algorithm has access, or learns a model of the environment. The model predicts transitions and rewards: the model allows the agent to plan. In case the agent is model free, the agent must learn the model from the environment.

2.2 Bonus Unit 1

It uses the ML-Agents Toolkit https://github.com/Unity-Technologies/ml-agents

2.3 Unit 2: Introduction to Q-Learning (a value-based method)

2.3.1 What is RL? Recap

The agent decision making process is called π : given a state, the policy outputs an action or a probability distribution over a set of actions. The goal is to find the optimal policy π^{*}, the policy that leads to the best expected cumulative reward. Two main types of RL methods:

- policy based methods: train the policy directly to find which the action to take in a given state. The policy is a neural network, there is no value function. The behavior of the policy is not defined by hand (implemented), the training defines it.
- value-based methods: train a function to learn which state is more valuable, and use this value function to take the action that leads to it. This section is about value-based methods

2.3.2 Two types of value-based methods

The value of a state is the expected return that the agent gets if it starts in that state and then acts according to the policy:

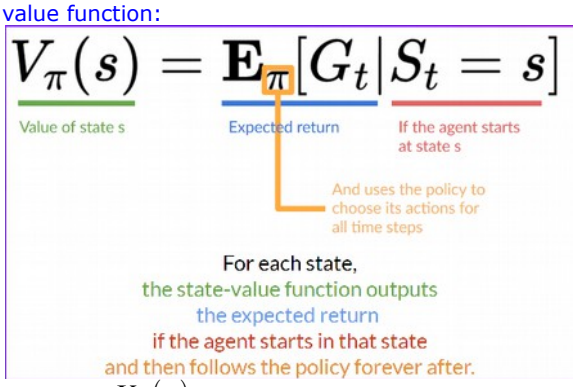
$$v_\pi(s) = E_\pi[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + ... + \gamma^k R_{t+k+1} + ... \mid S_t = s]$$

What does it mean "*according to our policy?*": in value based methods we do not have a policy, since we train a value function. Since the policy is not learned/trained, we need to specify its behavior. For instance, a **Greedy policy**: given the value function, the greedy policy takes the actions that lead to the biggest reward. The greedy policy is $\pi(s) = arg\ max_a Q_\pi(s,a)$

In value-based methods, finding the optimal value function leads to finding the optimal policy.

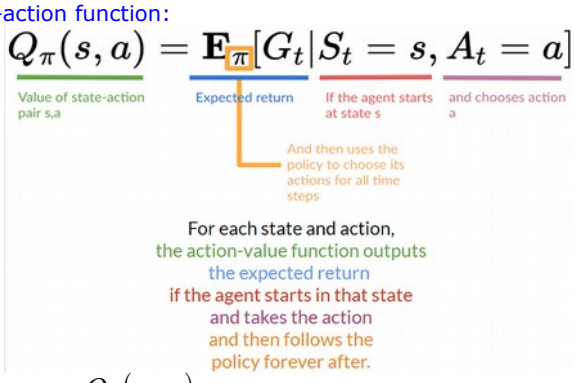
Relation between optimal value of a state and optimal policy $\pi^*(s) = arg\ max_a Q^*_\pi(s,a)$

State value function:



For each state, $V_\pi(s)$ provides the expected return if the agent starts in s and follows the specified policy π forever after.

State-action function:

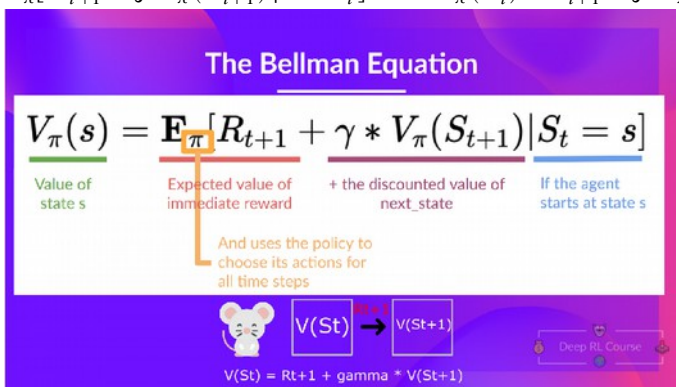


The function $Q_\pi(s,a)$ provides the expected return for each state/action pair, if the agent starts in s, executes a and follows the specified policy π forever after.

In both $V_\pi(s)$ and $Q_\pi(s,a)$ the output is the expected sum of all the rewards an agent gets if it starts in that state.

2.3.3 The Bellman Equation simplifies our value estimation

The Bellman equation: $V_\pi(s) = E_\pi[R_{t+1} + \gamma V_\pi(S_{t+1}) \mid s = S_t]$ => $V_\pi(S_t) = R_{t+1} + \gamma V_\pi(S_{t+1})$



The idea of the Bellman equation is that each state value is the sum of: immediate reward + discounted value of the next state.

2.3.4 Monte Carlo vs Temporal Difference Learning

Remember that an RL agent learns by interacting with its environment. Given the experience and the received reward, the agent updates its policy. Monte Carlo uses an entire episode before learning, Temporal Difference Learning uses only one SARS step:

$$(S_t, A_t, R_{t+1}, S_{t+1})$$

Monte Carlo approach: at the end of an episode, update each state using:

$$V(S_t) \leftarrow V(S_t) + \alpha[G_t - V(S_t)]$$

New value of state t | Former estimation of value of state t (= Expected return starting at that state) | Learning Rate | Return at timestep t | Former estimation of value of state t (= Expected return starting at that state)

At the end of each episode, we have a list of SARS.

G_t is not a reward, it is the total reward gained starting at time t.

$G_0 = R_1 + R_2 + R_3 + ...$; $G_1 = R_2 + R_3 + R_4 + ...$; etc.

$V_{next}(S_t) = V_{prev}(S_t) + \Delta V(S_t)$

$\Delta V(S_t) = \alpha(G_t - V_{prev}(S_t))$

At the beginning of the training, $V(S) = 0 \quad \forall S$

Temporal Difference Learning (TDL): learning at each step:

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

New value of state t | Former estimation of value of state t | Learning Rate | Reward | Discounted value of next state | TD Target

TDL estimates G_t with $G_t = R_{t+1} + \gamma V_{t+1}$, so:

$\Delta V(S_t) = \alpha(G_t - V_{prev}(S_t)) = \alpha(R_{t+1} + \gamma V(S_{t+1}) - V_{prev}(S_t))$

$G_t = R_{t+1} + \gamma V(S_{t+1})$ is called **TD target**.

This method is called **one-step TD**, thus TD(0).

TDL: at the end of a step we have: R_{t+1}, S_{t+1}, thus $TD = R_{t+1} + \gamma V(S_{t+1})$, and $\Delta V(S_t) = \alpha(TD - V_{prev}(S_t))$

In Monte Carlo we update $V_{next}(S_t)$ using the actual value of $G(S_t)$. In TDL we update $V_{next}(S_t)$ using an estimated return called TD target.

2.3.5 Introduction to Q-Learning. Q means Quality

(Link) Q-learning is an off-policy value-based method that uses a TD approach to train its action-value function $Q(s,a)$:

- off-policy
- Value-based method: the method learns a $Q(s,a)$ function.
- TD approach: upon each step the value function is updated, rather than at the end of the episode

Q-Learning is the algorithm that we use to train the $Q(s,a)$ function. Such $Q(s,a)$ provides the value of being in a particular state and taking a particular action at that state. $Q(s,a)$ is the quality of a when in the state s.

Difference between value and reward.

- The value of a state (or state action) is the expected cumulative reward if it starts in that state (or state-action pair) and it acts according to the policy
- The reward is the feedback it gets from the environment when is performs an action in a state.

The $Q(s,a)$ function is implemented as a table, where each cell corresponds to a state-action pair. For example, each row corresponds to a state, each column to an action. When the training is done, we have an optimal Q function, it means an optimal Q-table, so an optimal policy. Remember that $\pi^*(s)=arg\ max_a Q^*_\pi(s,a)$ optimal policy means optimal function.

2.3.5.1 The Q-learning algorithm is called Sarsamax

Input: α
$Q(s,a)$ is initialized with 0 values
For each episode **i**:
  $\epsilon = \epsilon_i$ // Selects the ϵ to use in the current episode.
  The first state is s_0
  $t=0$
  While Episode is not terminated:
    Choose the action a_i using the current $Q(s,a)$ and the ϵ greedy strategy
    Execute the action a_i, observe s_{i+1} and r_{i+1}
    Update $\Delta Q(s_i,a_i)=\alpha(r_{i+1}+\gamma\ max_a Q(s_{i+1},a)-Q(s_i,a_i))$
    $t=t+1$

The ϵ greedy strategy

- choose a random action with probability ϵ
- choose $max_a(Q(s,a))$ with probability $1-\epsilon$

At the beginning $\epsilon_0=1$: the action is chosen at random, to explore; ϵ_i is reduced as the training progresses.

Q update

$$Q(S_t,A_t) \leftarrow Q(S_t,A_t) + \alpha[R_{t+1} + \gamma max_a Q(S_{t+1},a) - Q(S_t,A_t)]$$

New Q-value estimation | Former Q-value estimation | Learning Rate | Immediate Reward | Discounted Estimate optimal Q-value of next state | Former Q-value estimation

TD Target

TD Error

Note: the TD target is calculated with the maximum Q given the state S_{t+1}, not the greedy policy.



$\epsilon=min_\epsilon+(max_\epsilon-min_\epsilon)e^{d*n}$

d : decay factor

n : the current number of episode

A typical decay is e.g. 0.0005.

Q-Learning is an off-policy because the policy used to select the next action is different from the policy to update the table (which is the greedy policy). An on-policy would use the same algorithm to select the next action and to update the value of Q.

Sarsa is an on-policy algorithm that uses the ϵ greedy for both the action selection and the Q update.

On-policy: same policy for acting and updating; **off-policy**: different policy for acting and updating.

2.4 Unit 3: Deep Q-Learning with Atari Games

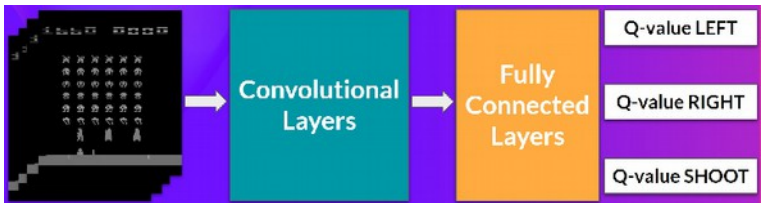
The RL-Zoo is used in the unit: https://github.com/DLR-RM/rl-baselines3-zoo

2.4.1 The Deep-Q-Network (DQN)

- Input: 4 frames as state
- Output: a vector of $Q(s,a) \quad \forall a \in A$

Algorithm:

- $Q(s,a)$ initialization
- ϵ greedy policy to select the action



2.4.2 The Deep Q-Learning Algorithm

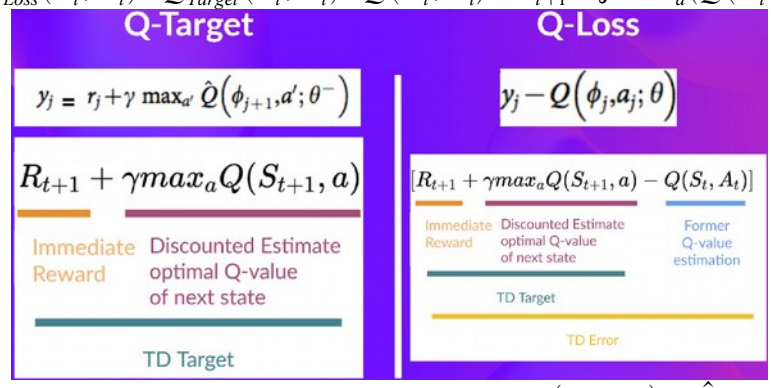
In Q-table, in every step an entry of the Q-table was updated using the TDL:

$\Delta Q(s,a) = \alpha(R_{t+1} + \gamma \, max_a(Q(S_{t+1}) - Q(s,a)))$

In Deep Q-Learning we create a loss function that compares out Q-value prediction with the Q-target and uses gradient descent to update the weights.

$\hat{Q}(S_{t+1}, a)$: means output of the model.

$Q_{Target}(S_t, A_t) = R_{t+1} + \gamma \, max_a(\hat{Q}(S_{t+1}, a))$ This is the value that the model should output when given the current state and action. Thus, loss is: $Q_{Loss}(S_t, A_t) = Q_{Target}(S_t, A_t) - \hat{Q}(S_t, A_t) = R_{t+1} + \gamma \, max_a(\hat{Q}(S_{t+1}, a)) - \hat{Q}(S_t, A_t)$



In the algorithm, Q depends on: the state, the action, the weights θ , thus: $Q(s,a,\theta)$. $\hat{Q}$:called Q-target

Initialize the replay memory D with capacity N
Initialize the action-state value function Q with random weights θ
Initialize the target action-state value function $\hat{Q}$ with weights $\bar{\theta} = \theta$
For each episode in 1, M :
  Initialize the sequence of the model inputs $s_1 = \{x_1\}$ // A sequence of states: images
  Initialize the preprocessed sequence $\phi_1 = \phi(s_1)$ // A sequence of preprocessed images.
  For $t \in [1,T]$:
    Sampling:
      ϵ greedy action: random action with probability ϵ ,otherwise select $a_t = argmax_a Q(\phi(s_t), a, \theta)$
      Execute the action a_t , keep track of: reward r_t and output image x_{t+1}
      ϕ_{t+1} is the preprocessed next state $\phi_{t+1} = \phi(s_{t+1})$
      Store $(\phi_t, a_t, r_t, \phi_{t+1})$ in D
    Training:
      Sample minibatch transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from D
      If episode terminates at step $j+1$ then $y_j = r_j$
                                                else $y_j = r_j + \gamma \, max_a(\hat{Q}(\phi_t, a, \bar{\theta}))$
      Perform gradient descent on $(y_j - Q(\phi_j, a_j, \theta))^2$ with respect to the parameters θ
      Every C steps set $\hat{Q} = Q$

Deep-Q training learning might e unstable because we combine Q-value and bootstrapping. To stabilize the training, we use:

- experience replay
- Fixed Q-target $\hat{Q}$
- Deouble deep learning

2.4.2.1 Experience replay

The replay memory D reuses the experience gained during training. The agent uses a memory of past experiences, so it does not forget past experiences as it learn. For example, when the agent gets to the second level, it can forget experiences of the first level. It also avoids actiion-values from oscillating. N Is the size of D : it is an hyperparameter of the algorithm.

2.4.2.2 Fixed target $\hat{Q}$

$\hat{Q}$ Is our current best estimation of the real Q .

2.4.2.3 Double DQN

In TDL we estimate the value of the next state with the highest Q. At the beginning of the training we do not have enough information about the best action to take, because Q is noisy. Maybe we do not select the best action. So we use:

- Q to select the best action
- $\hat{Q}$ to calculate the target value (TD target: estimation of the value of the next state after having executed an action)

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

New value of state t | Former estimation of value of state t | Learning Rate | Reward | Discounted value of next state | TD Target

2.5 Unit 3 Bonus Unit: Hyperparameter tuning with Optuna

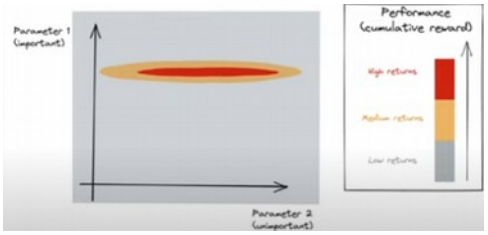
Automatic Hyperparameter tuning: improves the performance/training time. The tool is **Optuna**(https://optuna.org/).
Hyperparameter optimization, you usually try many configurations and wait that the training is done. Each configuration is given a budget. Different configurations are tested against a budget, then the more promising are tested with some more budget, etc. Until the optimal configuration is found, which is trained for 100%. There are two components:

- the **sampler** is the search algorithm, it decides wh ich configuration you need to actually try;
- the **pruner** (the scheduler): it decides when to stop a trial.

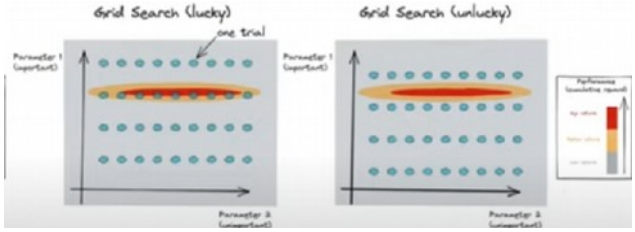
2.5.1 The Sampler

Samplers: the sampler uses

- **Performance landscape**: for example it is optimizing for parameter 1 and parameter 2 (parameter 1 is important, parameter 2 is unimportant, but you do not know it in advance). Parameter 1 is important and you have to choose it precisely, the other can be chosen loosely.
- **Grid search**: you test sets of parameters, parameters vary in fixed steps, but you can be lucky or not. Grid search searched parameters in a discrete space. You do not know how to discretize the space and the budget for each trial. Grid search scales poorly with the number of parameters. In literature, grid search not used as the baseline method. The baseline is random search, which is more competitive than grid search and it does not depend on the discretization. In Random search, you uniformely sample the parameter space.



Parameter



Grid search

Random search is another method. Uniformly sampling the parameter space: it is competitive with grid search.
Bayesian optimization, it tries to estimate the performance that you will get by choosing a set of parameters, it uses an algorithm like:

- Gaussian Process (GP)
- Tree of Parzen Estimators (TPE)

Black box optimization:

- Evolution strategies: (ES, CMA-ES)
- Particle Swarm Optimization (PSO)

2.5.2 Schedulers (Pruner)

It is the component that decides how much budget to allocate for each trial .
Median pruner: (used in Google Vizier). The algorithmis:

Prune if the trial's best intermediate result is worse than the median of intermediate results of previous trials at the same step.

Successive halving: you start with a set of promising configurations, and a minimum budget, then you discard the not promising and train the remaining for double of the budget, and so on. An advanced algorithm is called Hyperband.

2.5.3 Optuna

Hyperparameters tuning steps:

- **Define the search space**: the parameters to optimize and the ranges. Optuna supports integer variables, categorical variables, float (continuous variables). For example a categorical variable can be the activation function, that can be e.g. tanh or ReLU. Integer variable: the number of steps, e.g. between [64, 2048]. For continuous function, the variable can vary as logarithm.
- **Define the objective function**: objective function = what do you want to optimize for. Evaluation: you must pass a callback to the training library, calculate the performance, which is the reward in case of the RL. Then you notify the calculated performance to the scheduler, which decides whether the trial must be stopped.
- Choose a **sampler** and a **pruner**
- Define the model: sampling the model, sampling the hyperparameters, etc, training the model and sending to Optuna the final performance.
- Choose sampler, pruner and launch the study.

TPE sampler is bayesian, we start with a number of startup trials.

Common pitfalls:

- Most of the times Hyperparameter optimization is not needed (train longer first)
- Only you may find hyperparameters that are suited for the problem
- Noisy evaluation: in case you run twice with the same parameters, you get different results because the scene can be different. Do multiple evaluations , same agent with different random scenes; or take the 10 best trials.
- Search space is too wide or too small: an error can be setting a too-small or too big search space.
- Slow optimization:smaller budget: do optimization with a minimum budget
- Training can be not stable: manual tweaks

In RL, you usually want to maximize the mean episodic reward.

Recap: use automatic tuning when possible. Automatic tuning = sampler + pruner + objective function. Do not use grid search.

Note also the Physics engine EAGERx. https://github.com/eager-dev/eagerx

2.5.4 Tutorial

When compared with Supervised Learning, Deep Reinforcement Learning is far more sensitive to the choice of hyper-parameters such as learning rate, number of neurons, number of layers, optimizer ... etc.

Poor choice of hyper-parameters can lead to poor/unstable convergence. This challenge is compounded by the variability in performance across random seeds (used to initialize the network weights and the environment).

In the tutorial it sets a few constants, like N_TRIALS, etc.

Search space. A function, called e.g. `sample_a2c_params()`, defines the search space. It returns a dictionary of key → trial.suggest objects. It can also return more complex structures that contain suggested parameters.

Callback from Stable baselines. In order to integrate it with stable_baselines, it defines a callback that is called by stable_baselines, derived from `EvalCallback` and on every step it reports the performance (mean reward, index, etc) and if Optuna says so, it stops the RL training.

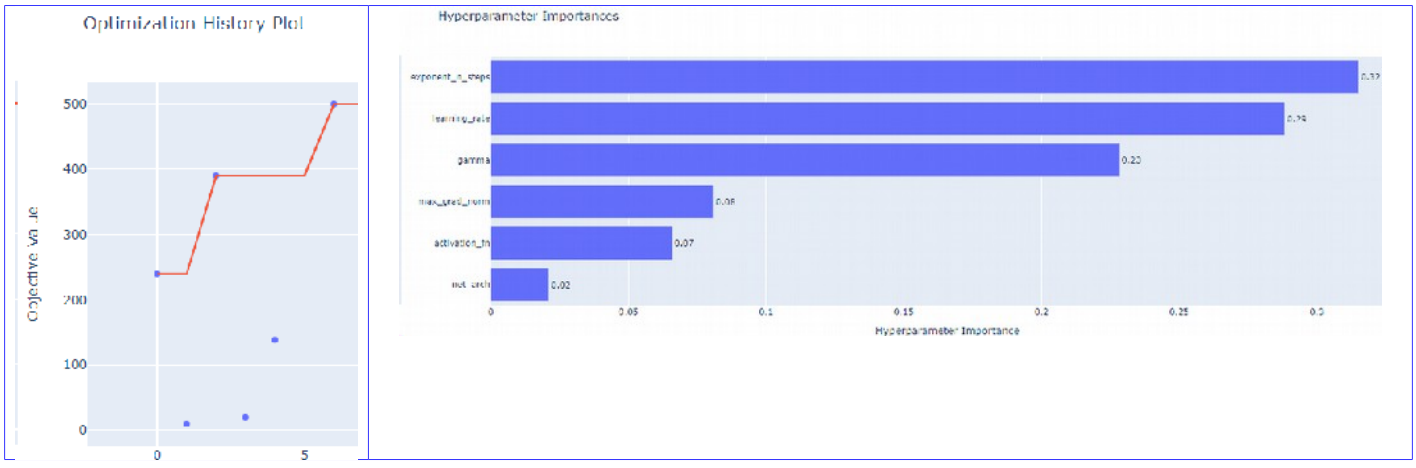
Objective function. What he calls objective function is a function that, given a Trial from Optuna:

- samples the hyperparameters
- creates the model, the environment and the Gym environment
- launches the training for the created model

The objective function is used by Optuna to evaluate one configuration.

Optimization loop: launches the optimization study and then it saves the results. This code is always the same.

When Optuna finishes a study, it can produce plots and tables like:



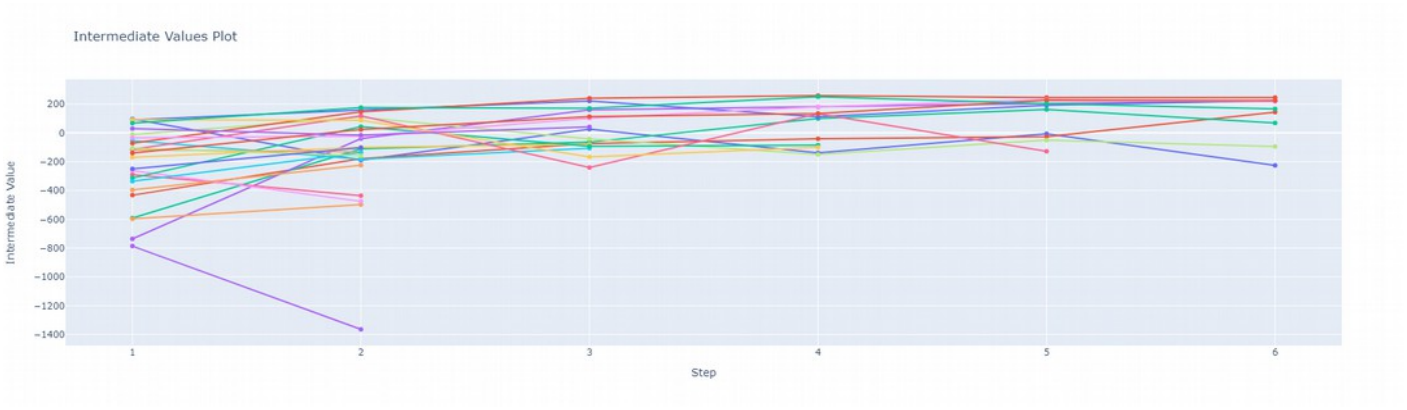


Illustration 1: Optuna: example of values plot

Link to the Notebook

2.6 Unit 4: Policy Gradient With PyTorch

2.6.1 Policy based and policy gradient methods

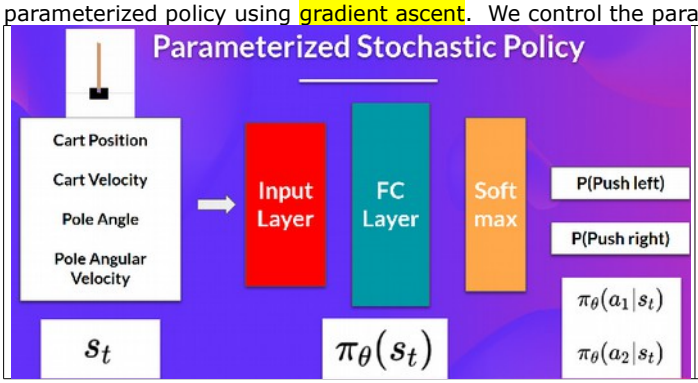
RL is based on the reward hypothesis: all goals can be described as the maximization of the expected cumulative reward.

Value-based methods. In the last units we studied value-based methods, where a value function gives the value of an action in a state. The value function is an intermediate step in finding a policy. $\pi^*(s) = arg\ max_a Q^*_\pi(s,a)$ The policy chooses the maximum value. In value-based methods, the policy exists because of the action value estimates, the policy is just a function. In policy based methods, we want to optimize the policy directly, without the intermediate value function.

What are policy based methods? The main goal of RL is finding the optimal policy π^*, that maximizes the expected cumulative reward.

Value-based methods: we learn a value function. The optimal value function leads to the optimal policy π^*. The objective is minimizing the loss between the predicted and target value, to approximate the true action-value function. The policy is implicitly generated from the value function. For example, it can be epsilon-greedy policy.

Policy based methods: we directly learn to approximate π^*, without a value function. We parameterize the policy. The policy π_θ depends on the parameters θ. The policy outputs a probability distribution over the actions (stochastic policy) $\pi_\theta = P[A|s;\theta]$ probability, given the state and the parameters. The objective is thus to maximize the performance of the parameterized policy using gradient ascent. We control the parameters θ that set the action distribution in a given state.



The policy $\pi_\theta(a|s)$ outputs a probability distribution over the actions. We optimize the policy π_θ, so that it leads to the best cumulative return. We define an objective function $J(\theta)$ that returns the expected cumulative return. We want to find the value θ that maximizes this objective function.

Difference between policy based and policy gradient methods. Policy gradient methods are a subclass of policy based methods. In policy-based methods the optimization is mostly on-policy: on each update we only use trajectories collected by our most recent version of π_θ.

- In policy-based methods, we search the optimal policy. We optimize the parameter θ indirectly by maximizing the local approximation of the objective function with optimization techniques like hill climbing, etc.
- In policy-gradient methods, we search directly the optimal policy, since it is a policy-based method. In policy gradient we optimize θ directly by performing gradient ascent on the performance of the function $J(\theta)$.

2.6.2 Advantages and disadvantages of policy gradient methods

Advantages of policy-graient methods:

- the policy is estimated directly
- policy gradient methods learn a stochastic policy, value based methods cannot. Consequences:
  - The exploration/exploitation trade-off is supported directly by policy gradient methods, with value-based methods we have to use the epsilon greed technique
  - policy gradient methods do not suffer of perceptual alising: the same observation sometimes needs a different action. Policy gradient methods have a stochastic component so that they do not always do the same sequence of actions. Policy gradient methods are more effective in high dimensional action spaces and continuous action spaces

In case of continuous actions: at inference time, Deep Q-Learning assigns a score to each action: finding the max vaue of a continuous output is itself an optimization problem.

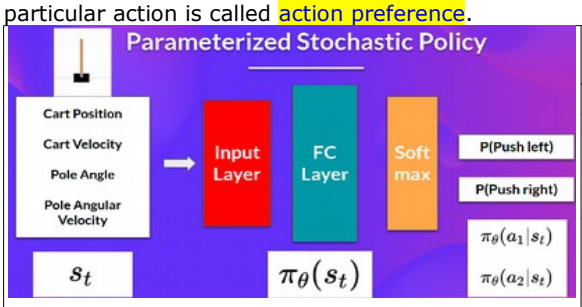
Policy gradient methods change smoothly over time. In value-based methods, the best action can change from left to right. In policy based methods the probability changes smoothly over time.

Disadvantages of the policy gradient methods:

- they can converge to a local minimum instead of a global minimum
- they take longer to train
- they can have high variance

2.6.3 Diving deeper into policy-gradient methods

The Big Picture. In policy-gradient methods, we try to find the parameters θ that maximize the expected return. We have a parameterized stochastic policy: a neural network that produces a probability distribution over actions. The probability of taking a particular action is called action preference.



CartPole-v1: the input is the state, the output is a probability distribution over the actions at that state.
The policy gradient tries to control the probability distribution of actions, so that good actions are choosen more frequently in the future. We let the agent interact with the environment during an episode. If we succeed, we consider that each action taken was good and must be sampled more frequently in the future, since it lead to success. In case of failure, we want to decrease the probability of taking that action when at that state.
High level policy gradient algorithm:

Training Loop:
Collect an **episode with the π** (policy).
Calculate the return (sum of rewards).

Update the weights of the π:
If positive return → **increase** the probability of each (state, action) pairs taken during the episode.
If negative return → **decrease** the probability of each (state, action) taken during the episode

Diving deeper into policy gradient methods π_θ Is parameterized by θ . $\pi_\theta(a,s)=\boldsymbol{P}[a|s;\theta]$ The policy function outputs a probability distribution over the actions, when in the state s .
To evaluate the policy, we define a score/objective function $J(\theta)$ over the parameters θ : θ affect $\pi_\theta(a,s)$ and $J(\theta)$.

$$J(\theta) = E_{\tau\sim\pi}[R(\tau)]$$

$$R(\tau) = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \gamma^3 r_{t+4} + \dots$$

Return: cumulative reward — Gamma: discount rate

Trajectory (read Tau)
Sequence of states and actions

The objective function $J(\theta)$ outputs an expected cumulative reward, by considering (*thoretically*) all trajectories. Each reward results from a trajectory τ , weighted by its probability $P(\tau;\theta)$.

$$J(\theta) = \sum_\tau P(\tau;\theta)R(\tau)$$

We calculate the expected return J(θ) by **summing for all trajectories**, the probability of taking that trajectory given θ and the return of this trajectory.

Probability of the trajectory (depends on θ since **it defines the policy that it uses to select the actions of the trajectory which as an impact of the states visited).**

Cumulative **return** from trajectory

$$J(\theta) = \sum_\tau P(\tau;\theta)R(\tau)$$

$$P(\tau;\theta) = \left[\prod_{t=0} P(s_{t+1}|s_t,a_t)\pi_\theta(a_t|s_t)\right]$$

Environment dynamics (state distribution)

Probability of taking that action a_t at state s_t

$J(\theta)$ is calculated over multiple trajectories. Each trajectory is defined by a sequence of state-action (s_t,a_t) pairs. The probability that a trajectory occurs is the product of, for each timestep, the product of the probability of switching into the next state and the probability of taking that action.

$\pi_\theta(a_t|s_t)$ depends on θ , $P(s_{t+1}|s_t,a_t)$ does not.

The objective is to find the parameters θ maximize the expected return(=cumulative reward over a trajectory).

$$\max_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}[R(\tau)]$$

Gradient ascent and the Policy-gradient Theorem

One step of the gradient ascent is: $\theta_{next} = \theta_{curr} + \alpha \nabla_\theta J(\theta)$, however:

- we cannot calculate the expected value over all possible trajectories τ , we calculate a gradient estimation with a sample based estimate

- We cannot differentiate $J(\theta)$ to calculate the gradient $\nabla_\theta J(\theta)$

Since $J(\theta)$ is not differentiable we can reformulate the gradient, Given the Policy Gradient Theorem

The Policy Gradient Theorem:

For any differentiable policy and for any policy objective function, the policy gradient is:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a_t|s_t) R(\tau)]$$

The Reinforce algorithm (Monte Carlo Reinforce)= Monte-Carlo policy-gradient is a policy gradient algorithm that uses an estimated return from an entire episode to update θ

The loop is (in case we update the policy after every episode):

- given the current policy π_θ , collect an episode τ
- Estimate the gradient $\nabla_\theta J(\theta) \approx \hat{g}$
- Update the policy $\theta = \theta + \alpha\ \hat{g}$

$$\nabla_\theta J(\theta) \approx \hat{g} = \sum_{t=0} \nabla_\theta \log \pi_\theta(a_t|s_t) R(\tau)$$

Estimation of the gradient (given we use only one trajectory to estimate the gradient)

Probability of the agent to select action at from state st given our policy

Direction of the steepest increase of the (log) probability of selecting action at from state st

Cumulative return

The loop in case we collect multiple episodes

$$\nabla_\theta J(\theta) \approx \hat{g} = \frac{1}{m} \sum_{i=1}^{m} \sum_{t=0} \nabla_\theta \log \pi_\theta(a_t^{(i)}|s_t^{(i)}) R(\tau^{(i)})$$

Estimation of the gradient (given we use some trajectories to estimate the gradient)

Scaling factor inversely proportional to the number of trajectories (m)

Probability of the agent to select action at from state st given our policy in trajectory (i)

Direction of the steepest increase of the (log) probability of selecting action at from state st

Cumulative return of i-th trajectory

Remember that $\frac{d \ln_a(x)}{dx} = \frac{1}{x \ln(a)}$ and $f'(g(x)) = f'(g(x)) g'(x)$

The Policy Gradient Theorem. Remember that:

$$J(\theta) = \sum_\tau P(\tau;\theta) R(\tau)$$

We calculate the expected return J(θ) by summing for all trajectories, the probability of taking that trajectory given θ and the return of this trajectory.

Probability of the trajectory (depends on θ since it defines the policy that it uses to select the actions of the trajectory which as an impact of the states visited).

Cumulative return from trajectory

$$J(\theta) = \sum_\tau P(\tau;\theta) R(\tau)$$

$$P(\tau;\theta) = [\prod_{t=0} P(s_{t+1}|s_t, a_t) \pi_\theta(a_t|s_t)]$$

Environment dynamics (state distribution)

Probability of taking that action a_t at state s_t

By the way: what are logits? They are the outputs of a neural network before the activation function is applied. They are the unnormalized probabilities of an item belonging to a certain class. Softmax is often applied to the logits to normalize the sum to 1.

The logits are the x_i values and the output of the SoftMax layer is $\frac{e^{x_i}}{\sum_j e^{x_j}}$ Moreover $\log_M(ab) = \log_M(a) + \log_M(b)$

For a single trajectory:

$$\nabla J(\theta) = R(\tau) P(\tau|\theta) \sum_{t=1}^{T} (\nabla \log \pi_\theta(a_t|s_t)) = E[R(\tau) \sum_{t=1}^{T} (\nabla \log \pi_\theta(a_t|s_t))]$$

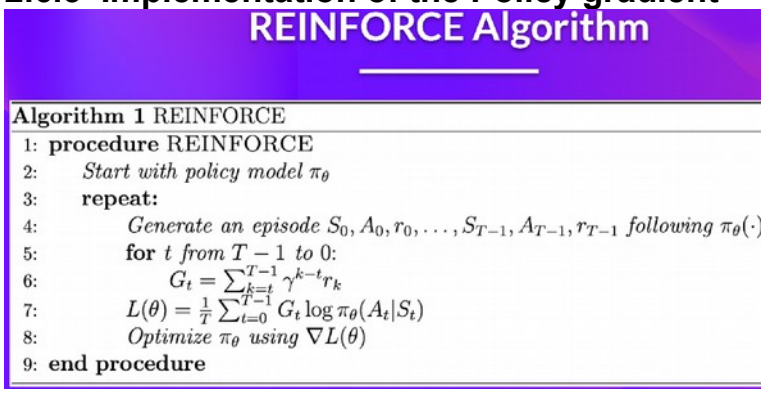
(probability $P(\tau|\theta)$ means expectation, so $P(\tau|\theta)$ disappears and on its place there is the $E[\,]$ that wraps). All trajectories have the same probability, so if there are m trajectories, the probability is $\frac{1}{m}$ (link).

$$\nabla J(\theta)=\frac{1}{m}\sum_{i=1}^{m}\sum_{t=1}^{T}\left(\nabla \log \pi_\theta\left(a_t^{(i)} \mid s_t^{(i)}\right)\right) R\left(\tau^{(i)}\right)$$

2.6.4 Intro to Policy Optimization at OpenAI

Link We consider a stochastic, parameterized policy. We aim to maximize the expected return $J(\theta)$ $\theta_{k+1}=\theta_k+\alpha \nabla_\theta J(\theta_k)$ we need an expression for the policy gradient that we can compute.

2.6.5 Implementation of the Policy gradient



Algorithm 1 REINFORCE

procedure REINFORCE
  Start with policy model π_θ
  repeat:
    Generate an episode $S_0, A_0, r_0, \ldots, S_{T-1}, A_{T-1}, r_{T-1}$ *following* $\pi_\theta(\cdot)$
    for t *from* $T-1$ *to* 0:
      $G_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_k$
    $L(\theta) = \frac{1}{T} \sum_{t=0}^{T-1} G_t \log \pi_\theta(A_t|S_t)$
    Optimize π_θ *using* $\nabla L(\theta)$
end procedure

3 CS285 Deep Reinforcement Learning as UC Berkley

Course Home page, Fall 2023 Youtube playlist Youtube playlist: Seen Up to Lecture 1: Part 1, 2,

3.1 Lecture 1:

In RL, data is not necessarily ground truth, but it provides experience. RL does no copy what is in the data, it uses the success failure labels to maximize the number of success.
RL is a mathematical formalism for learnng-based decsion making, an approach for learning decision making and control from experience. (LLM(s) work by estimating the probability mass).
In standard machine learning we assume to have X and Y, so we approximate Y=f(X), but we also assume that data is independent and identically distributed. Assumes that an output Y has no effect on other inputs, assumes the the f(x) that produced them is the same for all of them. We also assume known ground outputs in training.
RL s not iid data and ground truth is not known, as long as we know if we succeeded or failed.
RL provides a formalism for behavior.
RL stems from animal behavior and psychology; but also from control optimization and evolutionary algorithms.
RL: mathemtical formalism for learning based decision making and an approach for learning decision making and control from experience.
In RL you made a set of decisions along the way you do not know which of them caused you to fail, you only know success/failure.
Data driven AI: it learns the real world from data, but it does not try to do better than the data: it is all about using data
RL: optimizes a goal with emergent behavior, bit it needs to figure out how to use at scale : it is all about optimization.

Data without optimization does not allow us to solve new problems in new ways.

R.Sutton: "*two methods that seem to scale arbitrarily are learning and search*". Building the way we think does not scale in the long run. **Learning**: extracting patterns from data. **Searching**: using computation to extract inferences. Search is optimization= some optimization process that uses (typically iterative) computation to make rational decisions.
Searching allows us to understand the word. Searching leverages that understanding for emergence.
A bit of phylosophy: why do we need ML anyway? Why do we need brains? D.Wolpert "*to produce adaptable and complex movements. Movement is the only way we can affect the world*". We need ML to produce adaptable and complex decisions.
ML is seen as a problem of decision making, not a problem of prediction.

- Robot: how do I move my joints?
- Automonomous car: how do I steer?
- Image classification. What is the decision? The image label?

What other problems do we need to solve to enable real-world sequential decision making?
Where do rewards come from? In video games is obvious. Lazy Optimist: "*as human agents we are accustomed to operating with rewards that are so sparse that we only experience them once or twice in lifetime, if at all*". Learning can come from imitation. In brain, Basal ganglia, movement is close to reward.
Are there other forms of supervision?

- Learning from demonstrations
  - inferring rewards from observed behavior
- learning from observin the world: learning to predice, unsipervised learning
- Learning from other tasks: transfer learnin and metalearning

The idea that we predict the condequences of our motor commands has emerged as an important theoetical concept in all aspects of sensorimotor control. Prediction precedes control in motor learning. Moreover, using tools with predictine models.
Levesaring advances in pretrained models.
How do we build intelligent machines? Image you have to vuild an intelligent machine, where do you start? Learnng can be the basis of intelligence: some things we all do(e.g. Walking). Our learning mechanisms are likely powerful enough to do everything we associate with intelligence. A single algorithm to learn everything? What must the single algorithm do? Interpret sich sensory inputs and choose complex actions. In the brain, perceptions that ancitipate reward become assiciated with similar firing patterns asthe reward itself. What challenges still remain?

- We have methods that leasn from huge amount of data, and optimization methods in RL, but not methods that both use data and RL
- humans learn quickly, deep RL methods are slow
- humans reuse past knowledge, transfer learning in RL is an open problem
- it is not clear the role of the reward function and the prediction.

4 Reinforcement Learning Foundations

4.1 Getting Started in RL

Agent: reinforcement learning agents observe and explore the environment to learn.
State: position of the agent
Environment:
Action(s): the action is the agent choice of activity in a state
Reward: the score an agent gets for taking an action
Policy: the strategy for deciding the best action
Goal: the agent mission

4.1.1 A basic RL problem

The model of the environment can be unknown: if so, this is a model free prediction.
If the environment is known, this is a model-based prediction.
Model-free: you move to a new city, explore the city and learn new ways to navigate it every day.
Episodic tasks: they have a defined goal , when it is reached the session ends. Mostly solved with model-based methods.
Continuing tasks: these are tasks that don't have an end point; they continue forever. Mostly with model-free methods, because the environment maybe it is not totally understood.
The reward can come after every action or at the end of the episode.

4.1.2 Markov Decision Process

Variables include: States, Actions, Rewards, Environment, Discount Factor

- A set of States: S1 to St,
- A set of actions A1 to At
- Discount factor : gamma

Break down a problem into: a set of states, a set of actions, a set of rewards, a goal.

4.1.3 A basic RL solution

State value function: the expected total reward when starting from a particular state.
Action value function: the expected total reward for choosing a particular action when in a given state.
We want to take the best path that will give us the optimal reward.

4.2 RL Algorithms

4.2.1 Monte Carlo Method

Mostly used in episodic tasks. The agent performs many episodes, then compares them to determine the actions that work best. At the beginning, there is an equiprobeble random policy: the initial policy used by the agent without prior knowledge of the environment. The agent stores the rewards gained by running the equiprobable random policy.
All results: cumulative rewards for different states, are stored in a Q-table.

	State 1	State 2	State 3	State 4	State 5	State 6
Forward						
Backward						
Right						
Left						

The Q-table stores, for each state, the cumulative reward of taking an action from that state.
Q-table, for each action and state keep track of the accumulated reward. In the MCM the Q-table is updated at the end of every episode.

4.2.2 Temporal Difference Methods

The secon major category of RL methods. In TDM(s) the Q-Table is updated at every time-step.

4.2.3 Other RL algorithms

Deep Reinforcement Learning: use neural networks to map states and actions to rewards.
Inverse RL: the agent learns by imitation.
Multi-agent reinforcement learning: multiple reinforcement learning agents learning at the same time.

4.3 The Monte Carlo Method

Q-table: stores the state value for every action taken by an agent. For every action and state there is an expected cumuated reward, by applying the policy. A state for every column, an action for every row.
Exploration and exploitation. When an agent explores the environment, it tries to search the environment and to understand it, it tries to understand as much as it can as possible, it tries to reach many states and exploit information from them.
Exploration and exploitation are different but simultaneous activitirs performed by a RL agent. They are most useful in model-free systems. Can be used also in model-based systems to navigate the environment.
Optimal policy: in any state, the agent selects the action that leads to the best expected cumulative reward. This is not always correct: the agent can select other actions with a certain probability, because maybe they will lead to a better outcome. More

exploration can lead to better reward. So, this action with the new best reward can replace the previous action.
Best strategy to balance exploration and exploitation: initially favor exploration, then lead toward exploitation.
Monte Carlo Prediction uses the Q-table. Given a policy, how would the agent estimate the value function? The Monte Carlo prediction uses the Bellman equation to estimate the state and action value functions, known as expected cumulative reward. .

$$V_\pi(s) = E_\pi[R_{t+1} + \gamma V_\pi(S_{t+1}) \mid S_t = s]$$

Given the policy ∏
First visit and every visit MC prediction: Given the Q-table, a given entry is updated the first time we encounter a state-value pair, the second, third, etc. ? The every visit monte carlo, averages all the visits to a state to estimate its value.
The first visit prediction is better, the every visit prediction is seen as biased.
Monte Carlo Control: iterates between exploration (random policy) and exploitation to improve the policy.
Monte Carlo Control Cycle:

- Initialize the Q-table using equiprobable random policies
- Improve the policy using the Bellman equation
- Update the Q-table

New terms:

- greedy policy: it only selects the best action
- epsilon-greedy policies: sometimes the agent is given the opportunity to select other actions.
- Incremental mean: it is used to update the policy after every episode. Monte Carlo can be without incremental mean: the Q-table is updated after a few episodes; or it can be with incremental mean: the Q-table is updated after every episode.
- Constant Alpha: used to regulate the rate at which the agent learns. It is used to update the policy after every episode. Also known as learning rate.

4.4 The Temporal Difference Method

The TDM solves the equation at every step, for every state. It has low variance but it is biased. The MCM, on the contraty, it has high varance and low bias. The TDM is better if the probability depends on the immediate past state. TDM is mostly used for continuoing tasks.

4.4.1 SARSA

A form of TDM. SARSA: State, Action, Reward, State, Action. Initially the Q-table has random values.
SARSA use the current state value and the state value of the next to update the current value in the Q table.

Monte Carlo Method

$$Q(S_t, A_t) = Q(S_t, A_t) + \alpha(G_t - Q(S_t, A_t))$$

Temporal Difference – SARSA

$$Q(S_t, A_t) = Q(S_t, A_t) + \alpha(R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t))$$

4.4.2 SARSAMAX (Q-learning)

Uses the greedy policy.

SARSAMAX (Q-learning)

$$Q(S_t, A_t) = Q(S_t, A_t) + \alpha(R_{t+1} + \gamma max Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t))$$

In Monte Carlo and Sarsa, the policy is epsilon-greedy.

4.4.3 Expected Sarsa

The expected value of the state action is choosen.

Sarsa and expected Sarsa are on-policy difference algorithms
Sarsamax and Q-learning are called of policy Temporal Difference Algorithms.

5 Deep Reinforcement Learning [AP22], 414 pages

In AI, there are three categories of ML: supervised learning, unsupervised learning, RL (8).
Deep learning is approximating functions in high dimensional problems. RL methods learn from feedback provided by the environment.
(3) RL needs an environment providing feedback signals for the actions that the agent is trying. The agent chooses which actions to try. The agent can also stay in its comfort zone and never try any new action to receive a better feedback.
RL can find solutions for sequential decision problems, or optimal control problems (4). Main applications: robotics and games.
Judea Pearl work on casual reasoning (10)

(12)

6 Reinforcement Learning Algorithms with Python([AL19])

6.1.1 An Introduction to RL

RL: trial and error from active interactions with the environment. (9) RL deals with sequential decision making aimed at reaching a goal. An RL problem is constituted by a decision maker called **Agent** interacting with an **Environment**. The agent interacts with the environment in the form of an **Action**, that has an effect. The Environment results in a new **State** and a **Reward**. The agent goal is to maximize the reward accumulated during its lifetime.

- a_t the action at time t
- r_t the reward at time t . The reward can be dense, sparse, or delayed (12).
- $\sum_{i=0}^{t} r_i$ the sum of all rewards, the value that must be maximized

RL can deal with environments that are dynamic, uncertain and non-deterministic.
Comparing RL and supervised learning (12). In supervised learning the desired output taken from a fixed dataset, it is the best possible output. In RL there is no correct answer, only reward; focused on sequences of actions. The state depends on the agent sequence of actions. Exploration/exploitation dilemma.
History of RL: started in 1960s and 1970s: Markov decision process (**MDP**): a general framework to model decision-making in stochastic situations. Dynamic programming (**DP**) is a method for optimal control. In the 80s, temporal difference learning (**TD**). Tabular methods are not scalable when the state space is huge. Deep learning: deep Q network (DQN) (14).
Deep learning can tackle problems with a high dimensional state space.
Elements of RL (16). The **policy** is how the agent selects the next action. The policy can be deterministic (a single action is chosen) or stochastic (the policy provides a probability for each action) (17). On-policy: the policy that improves is the same as the one that acts on the environment. Off-policy: a policy acts on the environment (behavior policy), another learns but it is not used (target policy). Policy gradient method is a family of RL algorithms.
The cumulative reward in a given state is called value function. It is the long-term quality of the current state. The cumulative reward expected if the agent starts in a given state. The reward is the immediate performance, the value measures the performance in the long run. The value function can be a function of the state alone (state-value function), or a function of the state-action pair (action-value function, or value-function) (18). Some policy gradient methods use a value-function to learn the policy: they are called actor-critic methods
Reward. The reward is assumed to be part of the environment (19). There can be dense reward or sparse reward (20). Imitation learning and inverse RL methods deal with the absence of reward from the environment.
The **Model** is an optional part. The model predicts the next state and reward, given state and action. Model-based methods (20).
Applications of RL (20): automatic machines, optimization processes, control. The main fields where RL is already adopted are: gaming (self-play is where the agent is against itself), robotics to predict maintenance, etc.

6.1.2 Implementing the RL Cycle and OpenAI Gym

(27)

6.2 Reinforcement Learning – An Introduction ([SB18])

From [SB18][1] RL is learning what to do to maximize a numerical reward. The learner must discover which actions yield the most reward by trying them: trial and error search with delayed reward.

RL is different from supervised learning, where the agent is given labelled examples, and unsupervised learning, where the agent is given unlabelled examples but the task is finding the underlying structure in the data.

[3] In RL; tradeoff between **exploration** and **exploitation**. To obtain reward, the agent can exploit what gave reward in the past, but to know such actions, it has to explore, to try new actions. Both exploration and exploitation can lead to failure.

- **Policy**:[28] the agent's way of behaving, at a given time. The mapping from perception to action. The *stimulus-response* rules. Policy alone is sufficient to determine behavior.
- **Reward signal**: each time step, the environment sends a reward to the agent. The agent tries to maximize the total reward it receives over the long run. The reward is the primary basis for altering the policy. Reward is immediate.
- **Value function**. The value of a state is the total reward that an agent can accumulate in the future, starting from that state. Reward is the immediate pleasure, value is a more refined judgement of how much we are pleased in a given state.
- **Model of the environment**. It mimics the behavior of the environment. Model-based and model-free methods exist.

6.2.1 Tabular Solution Methods

[45] In simple cases, value functions can be described by tables. [47] RL is more about evalution than instruction.

- **Evaluation**. Evaluative feedback: how good an action was (even if it is the best possible)
- **Instruction**. Instructive feedback: how good an action is in absolute terms, independently of the action actually taken.

6.2.1.1 K-Armed bandit problem

The agent faces a repeated choice among k possible choices or actions. After a choice there is a numerical reward depending on the action, chosen from a stationary prob-distribution. [48] Each of the k actions has an expected reward, called *value* of the action. Each action has a value: it can be evaluated by the action-value function. A_t is the action taken on time step t.

The expected reward of an action a, its **value**, is:

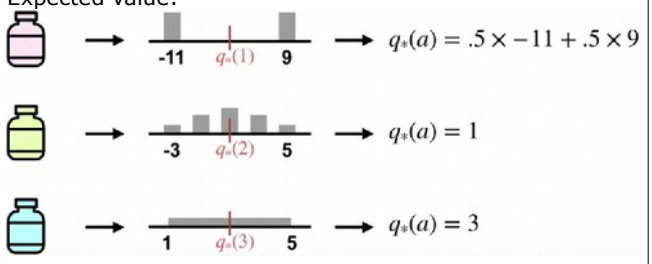
$$\forall a \in \{1,2,\dots,k\} \quad q_*(a) = E[R_t \mid A_t = a] = \sum p(r \mid a) * r$$

It is unknown for every action. The agent goal is to maximize the expected reward.

$\sum p(r \mid a) * r$: given an action, each possible reward from that action is weighted by the probability of that reward.

The sum can become the integral in the continuous case.

Expected value:



$Q_t(a)$: the estimated value of the action a at the time t. Actions with the greatest $Q_t(a)$ are the *greedy* actions.

Selecting greedy actions means **exploiting** the current knowledge of the value of the actions. Selecting a non-greedy action means **exploring**, because it improves the value of non-greedy actions. Exploration produces greater reward in the long run.

6.2.1.2 Action-value Methods

Action-value methods [49] are methods to estimate action values and using those estimates to select actions. The next action is selected by evaluating only the possible actions: *the environment is not taken into account*. There is no state and context.

$$Q_t(a) = \frac{\text{sum of rewards when } a \text{ was taken prior to } t}{\text{number of times } a \text{ was taken prior to } t} = \frac{\sum_{i=1}^{t-1} R_i * 1_{A_i = a}}{\sum_{i=1}^{t-1} 1_{A_i = a}}$$ = average reward when a was chosen.

$1_{predicate}$ is 1 if $predicate$ is true. If the denominator is zero $Q_t(a)$ gets a default value, e.g.zero. As t goes to infinity, $Q_t(a) \to q_*(a)$ .This is called *sample-average* method of action selection (*it:media campionaria*).

- **Greedy action selection** $A_t = argmax_a Q_t(a)$: it exploits current knowledge to maximize immediate reward.
- ϵ **-greedy action selection**: with small probability ϵ, the action is selected randomly among all actions, with equal probability (*near-greedy*).

The greedy action is the action that has the largest current estimated value.

[50] With ϵ greedy , if there are $n > 0$ actions, the greedy action is selected with probability $(1-\epsilon) + \frac{\epsilon}{n} > (1-\epsilon)$

[53]How to efficiently estimate Q_{n+1} without storing in memory every R_i received? $Q_n = \frac{1}{n-1} \sum_{i=1}^{n-1} R_i$

$$Q_{n+1} = \frac{1}{n} \sum_{i=1}^{n} R_i = \frac{1}{n} \left(R_n + \sum_{i=1}^{n-1} R_i \right) = \frac{1}{n} \left(R_n + (n-1) \frac{\sum_{i=1}^{n-1} R_i}{n-1} \right) = \frac{1}{n} \left(R_n + (n-1) Q_n \right) = Q_n + \frac{1}{n} \left(R_n - Q_n \right)$$

$$\text{NewEstimate} = \text{OldEstimate} + \text{Step} * [\text{Target} - \text{OldEstimate}]$$ [53]

I would call it $$\text{NewEstimate} = \text{OldEstimate} + \text{Step} * [\text{MeasuredValue} - \text{OldEstimate}]$$

$$\text{NewEstimate} = \text{OldEstimate} + \text{Step} * \text{EstimationError}$$ In general $Step = \alpha_t(a)$.

Non-stationary problem the action values, that is the distribution of rewards, change with time[54]

$0 < \alpha \leq 1 \qquad Q_{n+1} = Q_n + \alpha(R_n - Q_n) = \alpha R_n + \alpha(1-\alpha) R_{n-1} + \alpha(1-\alpha)^2 R_{n-2} + \ldots$

$Q_{n+1} = (\sum_{i=0}^{n-1} \alpha(1-\alpha)^i R_{n-i}) + (1-\alpha)^n Q_1$ The sum of the weights is one. Called *exponential recency-weighted average*.

[55] The step might vary from step to step $\alpha_n(a)$: the step at iteration from n to $n+1$ when a was selected.

The step converges to the true action if: $\sum_{n=1}^{\infty} \alpha_n(a) = \infty$ and $\sum_{n=1}^{\infty} \alpha_n^2(a) < \infty$

The conditions are met for $\alpha = \frac{1}{n}$ but not for $\alpha_n(a) = \alpha$ but this is desirable in non stationary environments.

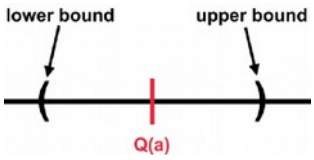
Optimistic initial values[56]: the methods discussed so far are biased by the initial estimation of the actions: $Q_1(a)$.

The initial value must be chosed and it is a way to supply some prior knowledge about the expected action rewards.

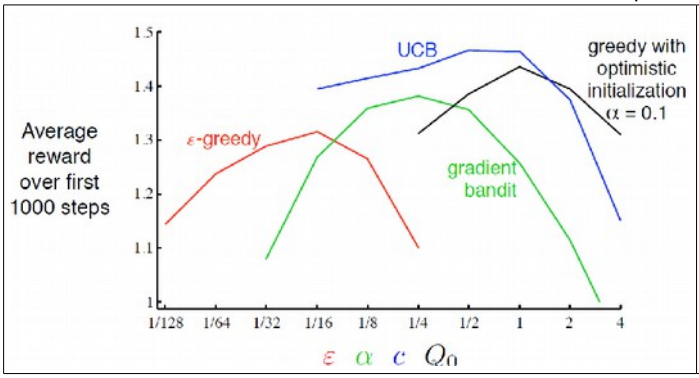
We can set the initial value to an high value to encourage exploration: this technique is called optimistic initial values, effective for stationary problems, not for non stationary problems where it is necessary to explore later.

All methods that treat initial conditions in a special way (opt.init.values, sample-value, etc.) do not help with the non stationary. Moreover, in some cases we may not know what is the maximum reward, so we cannot set an optimistic initial value.

Upper-Confidence-Bound Action selection[57] it is better to select, among the non-greedy actions, those that have the highest potential of being optimal: UCB: $A_t = argmax[Q_t(a) + c\sqrt{\frac{\ln t}{N_t(a)}}]$: the sqrt-term measures the uncertainty, or variance, in estimating a 's value.

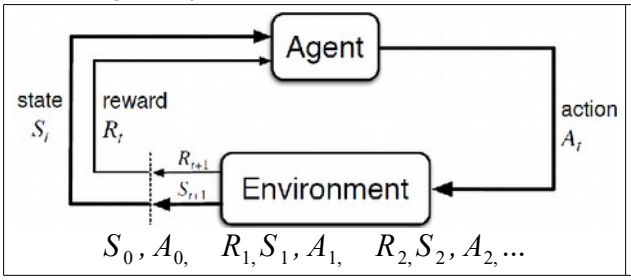
Uncertainty in the estimate Given an action, its action value may be known with a given confidence. Q(a) is the current estimate for action **a**.		The confidence interval has an upper and lower bound. We are thus confident that the real q(a) lies in the confidence interval. If the interval is small, we are certain of our estimate.
$A_t = argmax[Q_t(a) + c\sqrt{\frac{\ln t}{N_t(a)}}]$		The first term is the exploitation part, the second the exploration part. c controls the amount of exploration.

From the video: Contextual bandits in Real-World RL Shift priorities https://hunch.net/~rwil/

	**Comparison of the different algorithms** [64] all algorithms depend on one or more parameters. It is possible to measure the area under the learning curve (reward over time). For each algorithm, the area is plotted against the parameter value. This graph is called *parameter-study*. The X-axis is in logaritmic scale. Most algos have a U-shaped curve: the perform best at an intermediate value of the range.

6.2.2 Finite Markov Decision Processes (MDP(s))

[69] Finite MDP: it has evaluative feedback, but also associative aspects (actions depend on situations). In k-bandits we estimated $q_*(a)$, in MDPs we estimate $q_*(a, s)$ the value of an action a in a state, where s is a state, or $v_*(s)$ the value of a state *given optimal action selection*.

	The agent (learner / decision maker) interacts with the environment. Continuous, step-wise interaction between Env and Agent. Sequence of time steps. At each step t : • Agent receives a representation of the Env state $S_t \in \boldsymbol{S}$ • the Agent selects on action allowed in S_t : $A_t \in \boldsymbol{A}(s)$ • next step: the Env gives a reward $R_{t+1} \in \boldsymbol{R}$ and updates

	the state to S_{t+1}

Finite MDP: $\boldsymbol{R}, \boldsymbol{A}, \boldsymbol{S}$ have a finite number of elements. Note that also $\boldsymbol{R}$ has a finite number of elements. Actions are not probabilistic, States and Rewards are propabilistic. Situations=States. There is a trajectory of experience $\{S_t, R_t, A_t\}$

$p(s', r \mid s, a) = Pr(S_t = s', R_t = r \mid S_{t-1} = s, A_{t-1} = a) \in [0,1]$ The probability that the Env goes in a state and gives a reward, given the previous state and the previous action. $p()$ defines the dynamics of the MDP, it yields a probability value given four arguments. Note that $\sum_{s' \in S, r \in R} p(s', r \mid s, a) = 1 \quad \forall s \in S, a \in A(s)$ The probability of the next S_t and R_t only depends on the current state and action, not on something that happened before. The state must include information about all aspects that can make a difference for the future. It means that remembering past states will not improve our predictions about the future. From $p()$ we can determine:

- the probability of the next state given s, a : $p(s' \mid s, a) = \sum_{r \in R} p(s', r \mid s, a)$
- the expected reward given s, a : $r(s, a) = E(R_t \mid S_{t-1} = s, A_{t-1} = a) = \sum_{r \in R} r \sum_{s' \in S} p(s', r \mid s, a)$
- the expected reward given s', s, a : $E(R_t \mid s', s, a) = \sum_{r \in R} r \frac{p(r, s' \mid s, a)}{p(s' \mid s, a)} = \frac{\sum_{r \in R} r\, p(r, s' \mid s, a)}{\sum_{r \in R} p(r, s' \mid s, a)}$

Conditional probability: $P(A \cap B) = P(A \mid B) P(B) \rightarrow P(A \mid B) = \frac{P(A \cap B)}{P(B)} \quad P(A \mid B, C, D) = \frac{P(A \cap B \mid C, D)}{P(B \mid C, D)}$

The MDP framework is flexible: time-steps need not to be fixed intervals of time, states and actions can be high/low level.
Anything that cannot be changed arbitrarily by the agent is considered as part of the environment[72]. Even if the agent knows how the reward is calculated, R is still part of the environment. The agent/env interface is a boundary of control, not of knowledge. [72] The MDP framework is an abstraction of the problem of goal-directed learning from interaction.
All is reduced to the signals R,A,S passed between the agent and the environment. [73] States and Actions can be vectors, whereas Reward is always a single number. The sum of the probabilities of the next states following a given action is one.

The agent goal is formalized as the **reward**. The agent tries to maximize the expected accumulated rewards $E(\sum R_t)$.

[75] *All we mean by goal and purposes can be well thought as the maximization of the expected cumulative sum of the received scalar signal*. The reward must express *what* we want the agent to accomplish, not *how*.
Michael Littman: "*Intelligent behavior arises from the actions of an individual seeking to maximize its received reward signals in a complex and changing world*". How to design the goal? If there is a goal state, reaching such state gives reward=+1. If there is an undesirable state, reaching that state gives reward=-1. Inverse reinforcement learning: an agent is given an example of a task, the agent tries to come-up with the reward function that was maximized to act as in the example.

The return G_t is a function of the R_t sequence, for example: $G_t = \sum_{i=0}^{T} R_{t+i}$, G_t is the sum of R from that time to the final time step T . G_t is a stochastic variable. This makes sense when it is possible to think of trials/episodes (the episode finishes at the step T).

Episodic tasks. Each episode ends in a special terminal state, followed by a reset and a restard from a starting state or a distribution of states. Each episode is independent from the others. T is a random variable.

Tasks that are not episodic are called **continuing tasks**: they cannot be break into episodes, there is no terminal state. The return function cannot be expressed as before, because $T = \infty$. Example a thermostat, or an industrial process.

Discounting: rewards are discounted: $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots = \sum_{k=0}^{\infty} \gamma^k R_{k+t+1}$ where γ is the discount rate.

A reward received k steps in the future is worth γ^{k-1} . If $\gamma < 1$ and R_k is bounded, then G_t is finite. Discounting is necessary in continuing tasks.
Note: $G_t = R_{t+1} + G_{t+1}$

Unified notation for episodic and continuing tasks[79]: $S_{t,i}$: time t episode i . We can treat everything as continuing, and considering the episode termination as entering a special absorbing state where $R_t = 0$. Once entered, the agent never leaves the absorbing state.

6.2.3 Policies and value functions

[80] RL algorithms estimate value functions: how good is for the agent to be in a given state; depending on the expected rewards. The rewards depend on the chosen actions; that in turn depend on the policy.

Deterministic policy: the policy maps each state to an action, in case of deterministic policy: $\pi(s) = a : S \rightarrow A$ it is a table.

Stochastic policy:the policy $\pi(a \mid s)$ is the probability of selecting the action a when in s . $\pi(a \mid s): A \times S \rightarrow [0,1]$

Note: a policy that alternates between actions is not a valid policy: the state here should include something to keep track of the previous action. This is conditional on the action, not on the state (to do that, the state must include the information about the last action). Policies must depend only on the state.

Reward is the short-term gain. Objective is learning a policy that achieves the most reward in the long run, captured by v_π .

State-value function for the policy π: $v_\pi(s) = E_\pi[G_t \mid S_t = s] = E_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{k+t+1} \mid S_t = s\right]$ expected return from a given state, under policy π .

Value of taking the action under the policy: $q_\pi(s,a) = E_\pi[G_t \mid s = S_t, a = A_t] = E_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{k+t+1} \mid S_t = s, a = A_t\right]$

The agent could keep track of the rewards when following the actions in a given state, the average converges to $v_\pi(s)$. If the agent keeps track of the rewards gained by following each action in each state, the average converges to $q_\pi(s,a)$. These methods of estimating by averaging many samples are called *Monte Carlo methods* [81]. Other methods are based on parameterizing v_π and q_π and adjusting the parameters.

Bellman equation

: $\forall s \in S \quad v_\pi(s) = E_\pi[G_t \mid S_t = s] = E_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{k+t+1} \mid S_t = s\right]$

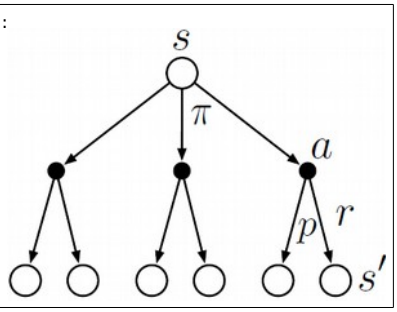
$= \sum_{a \in A} \pi(a \mid s) \sum_{s'} \sum_{r} p(s', r \mid s, a)[r + \gamma v_\pi(s')]$

$= \sum_{a \in A} \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a)[r + \gamma v_\pi(s')]$. It is an expected value.

B.eq.:averages all actions, each weighted by its probability of occurring.

Expresses as the value of a state depends on the value of the successor states.

Backup diagram:



Backup diagrams: open circles are states, filled circles are actions when in states (state-action pair). There are a number of ways to compute, approximate and learn $v_\pi(s)$. In backup diagrams, a state can appear as one of its successors. The arc from action to the new state encodes the reward and the new state.

Optimal policies and optimal value functions[84] a policy π is better or equal to π′ :

$\pi \geq \pi'$ if an only if $\forall s \in S : v_\pi(s) \geq v_{\pi'}(s)$ Being better defines a partial ordering over the policies.

Optimal policy: the policy that is better or equal to the others. There can be more than one optimal policy. There is always at least one optimal policy: it can be build by combining policies. The combining policy can select, given a state, the policy that has the highest state-value function.

π_* : the **optimal policies**. There can be more than one optimal policy.

All optimal policies share the same state-value function, called **optimal state-value function** $v_*(s) = max_\pi(v_\pi(s)) \forall s \in S$.

All optimal policies share the same **optimal action-value function** $q_*(s,a) = max_\pi(q_\pi(s,a))$

$q_*(s,a)$: the expected return for taking the action a in the state s , and thereafter following an optimal policy.

$q_*(s,a) = E[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s \quad A_T = a]$ The optimal action-value function gives the expected rewards after committing to a specific next action a when in a state s . The first action a does not depend on the policy: the next actions do.

$v_* = max_{a \in A} q_{\pi_*}(s,a) = max_{a \in A} E_{\pi_*}(G_t \mid S_t, A_t) = max_{a \in A} E_{\pi_*}(R_{t+1} + \gamma G_{t+1} \mid S_t, A_t)$

$= max_{a \in A} E_{\pi_*}(R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t, A_t) = max_{a \in A} \sum_{r, s'} p(r, s' \mid s, a)[r + \gamma v_*(s')]$

q_* written in terms of v_* : $q_*(s,a) = E[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a]$

The Bellman eq hold for each policy, including the optimal policy.

Recall that

$$v_\pi(s) = \sum_a \pi(a \mid s) \sum_{s'} \sum_r p(s', r \mid s, a)[r + \gamma v_\pi(s')]$$

$$v_*(s) = \sum_a \pi_*(a \mid s) \sum_{s'} \sum_r p(s', r \mid s, a)[r + \gamma v_*(s')]$$

$$v_*(s) = \max_a \sum_{s'} \sum_r p(s', r \mid s, a)[r + \gamma v_*(s')]$$

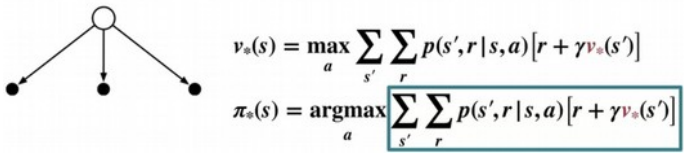
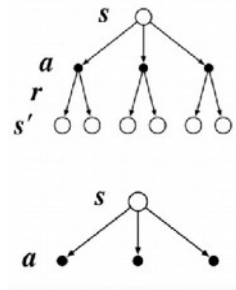
v_* cannot be derived with linear algebre, because max is not a linear function.

We don't know π_* , so we cannot derive v_* .

Bellman optimality equation for v*	

Bellman eq for v_* [85]: $v_*(s)=max_{a\in A}q_{\pi_*}(s,a)$ = $max_{a\in A}(E\left[R_{t+1}+\gamma v_*(S_{t+1})\,|\,S_t=s,A_t=a\right])$
= $max_{a\in A}(\sum_{r,s'}p(r,s'\,|\,s,a)[r+\gamma v_*(s')])$ For finite MDP(s), v_* has a unique solution [86].

Bellman eq for q_* [85]: $q_*(s,a)=E\left[R_{t+1}+\gamma\, max_{a'\in A}(q_{\pi_*}(S_{t+1},a'))\,|\,S_t=s,A_t=a\right]$
= $\sum_{r,s'}p(r,s'\,|\,s,a)[r+\gamma max_{a'\in A}q_*(s',a')]$ q_* in terms of the next q_*

Determining an Optimal Policy  $v_*(s) = \max_a \sum_{s'}\sum_r p(s',r\,|\,s,a)[r+\gamma v_*(s')]$ $\pi_*(s) = \operatorname{argmax}_a \sum_{s'}\sum_r p(s',r\,|\,s,a)[r+\gamma v_*(s')]$	[86]Once v_* is found, for each state, π_* must assign non-zero probability to the actions the move the state to states that have the maximum $v_*(s')$: a one step search. In other words, any policy that is greedy with respect to v_* is an optimal policy. (v_* takes into account the consequences of the actions). It is a greedy policy that is optimal in the long-run.
Determining an Optimal Policy  $\pi_*(s) = \operatorname{argmax}_a \sum_{s'}\sum_r p(s',r\,|\,s,a)[r+\gamma v_*(s')]$ $\pi_*(s) = \operatorname{argmax}_a q_*(s,a)$	To determine the optimal policy, if v_* is known, for each action the p must be computed.

With q_* the optimal policy if selecting the action that maximizes $q_*(s,a)$: the additional cost, with respect to v_* , is keeping track of all the costs of the state-action pairs.
[88]Solving the Bellman eq(s) is one route to finding the π_* . To do that:

- we must know the dynamics of the environment
- we must have enough computational power
- Markov property for states

Optimality and approximation [89] usually, the optimal policy can be generated, but it has unreacheable computational costs for some problems; the amount of available memory is another constraint. We must settle for approximations. There opportunities to achieve useful approximations: for example by not computing value for states that have low probability to occur.

6.2.4 Dynamic Programming

[95] **Dynamic Programming** (DP): a collection of algorithms to compute optimal policies, given a complete model of the environment as finite MDP: S , A , R are finite, $p(r,s'\,|\,s,a)$ is known.

$v_*(s) = \max_a \mathbb{E}[R_{t+1}+\gamma v_*(S_{t+1}) \mid S_t=s, A_t=a]$ $= \max_a \sum_{s',r} p(s',r\,|\,s,a)\left[r+\gamma v_*(s')\right]$, or $q_*(s,a) = \mathbb{E}\left[R_{t+1}+\gamma \max_{a'} q_*(S_{t+1},a') \;\middle|\; S_t=s, A_t=a\right]$ $= \sum_{s',r} p(s',r\,|\,s,a)\left[r+\gamma \max_{a'} q_*(s',a')\right]$,	Once we have found the optimal value functions, we can obtain optimal policies. DP algos turn Bellman eqs into assignments= update rules that improve the approximation of value functions.

6.2.4.1 Policy evaluation (Prediction)

Prediction problem = policy evaluation = how to compute $v_\pi(s)$
$v_\pi(s)=E\left[G_t\,|\,S_t=s\right]=E\left[R_{t+1}+\gamma v(S_{t+1})\,|\,S_t=s\right]=\sum_{a\in A}\pi(a\,|\,s)\sum_{r,s'}p(r,s'\,|\,s,a)\left[r+\gamma v(S_{t+1})\right]$

It is a system of $|S|$ equations in $|S|$ unknowns $v_\pi(s)$.

Iterative solution: $v_0, v_1, v_2, v_3, \ldots$ is a sequence of approximate value functions, each mapping $S^+ \to \Re$. Each v_i is a

$S^+ \to \Re$. S is the set of states; S^+ Is the set of states, including the terminal state that has value zero.
Iterative approximation using Bellman: $v_{k+1}(s) = \sum_{a \in A} \pi(a \mid s) \sum_{r,s'} p(r, s' \mid s, a)\left[r + \gamma v_k(s')\right]$:iterative policy evaluation
$v_{k+1}(s)$ is the next approximation of the state s ; $v_k(s')$ is the previous approximation of the successive state s' .
Moreover:

- $v_k = v_\pi$ is a fixed point for the update rule
- $\lim_{k \to \infty} v_k = v_\pi$ under the same conditions that guarantee the existence of π

The iterative policy evaluation above is called *expected update*. The expected immediate rewards of the transition from a state to all the next states are used.
[97]When implemented in a computer program two arrays of one array can be used. One array = in place update. The in-place version converges faster. The in-place version is preferred.
[98]

7 Biography

ID	Title	Version
[AL19]	"*Reinforcement Learning Algorithms with Python - Learn, understand, and develop smart algorithms for addressing AI challenges*", Andrea Lonza, Packt Publishing, 356 pages. Code on gitHub: https://github. com/PacktPublishing/Reinforcement-Learning-Algorithms-with-Python	1st Edition, 2019
[AP22]	"Deep Reinforcement Learning", Aske Plaat, Springer, 414 pages.	1st Edition, 2022
[SB18]	"Reinforcement Learning – An Introduction", R.S.Sutton, A.G.Barto, MIT Press.	2nd Edition, 2018

https://sites.google.com/view/deep-rl-bootcamp/lectures
https://rail.eecs.berkeley.edu/deeprlcourse/
https://www.davidsilver.uk/teaching/
https://stable-baselines.readthedocs.io/en/master/guide/rl.html